

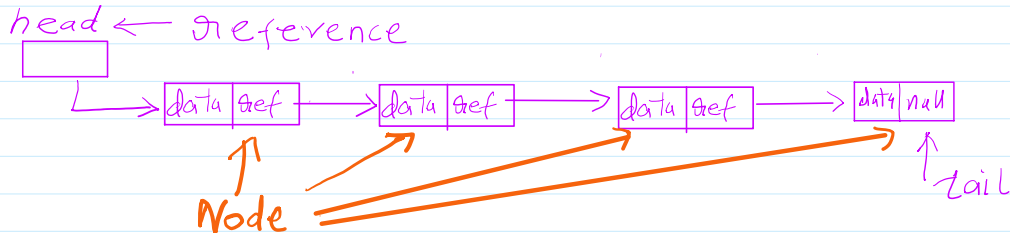
Linked List Data Structure

10 August 2026 19:52

A linked list is a linear data structure that includes a series of connected nodes. A linked list can be defined as a set of nodes that are randomly stored in memory. A node in the linked list contains 2 parts, that is the first is the data part and 2nd is the reference part. The last node of the list contains a pointer to NULL. After array linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

Representation of a linked list

A linked list can be represented as a connection of nodes in which each node points to the next node of the list.

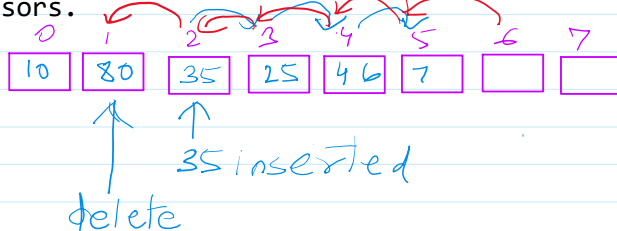


Till now we have been using an array data structure to organize a group of elements that are to be stored individually in the memory. However, array has several advantages and disadvantages that must be known to decide the data structure that will be used throughout the program.

Why use a linked list over an Array?

Structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays:

- The size of the array must be known in advance before using it in the program.
- Increasing the size of array is a time consuming process. It is almost impossible to expand the size of the array at runtime.
- All the elements in the array need to be contiguously stored in the memory. Inserting or deleting an element in the array meets shifting of all its predecessors.



The linked list is useful because:

- It allocates the memory dynamically, all the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of reference variables.
- In a linked list, size is no longer a problem since we do not need to define its size at the time of declaration. The list grows as per the programs demands, and it reduces its size as per requirement.

Structure of a Node

```
package DataStructures.LinkedListInJava;
```

```

/**
 * Write a description of class Node here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Node
{
    private int data;
    private Node next;

    public Node(int data){
        this.data = data;
        next = null;
    }

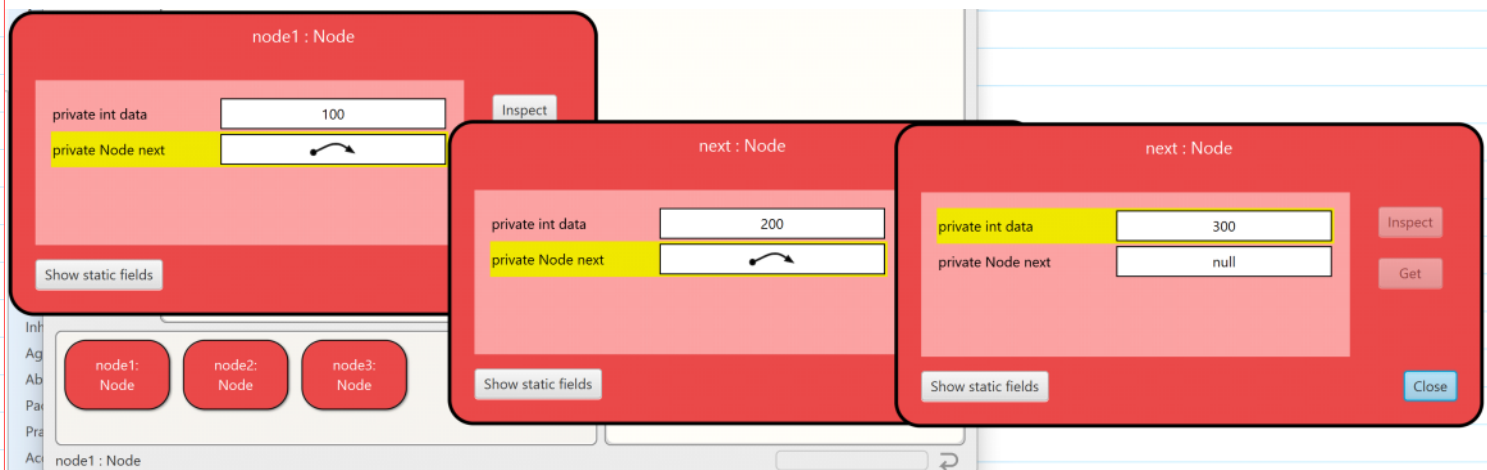
    public int getData(){
        return data;
    }

    public void setData(int data){
        this.data = data;
    }

    public void setNext(Node n){
        this.next = n;
    }
    /**
     * @return Returns the current node reference
     */
    public Node getNode(){
        return next;
    }

    public String toString(){
        return "==>"+data+" ";
    }
}

```



```

Node head = node1;
while(head != null){
    System.out.print(head);
    head = head.getNode();
}

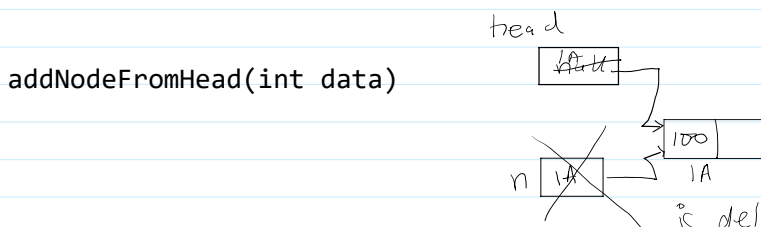
```

Output:
 ==>100 ==>200 ==>300

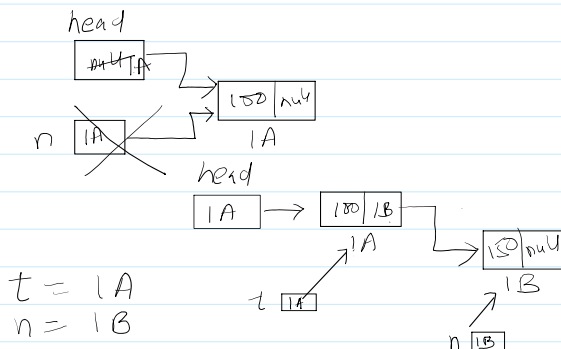
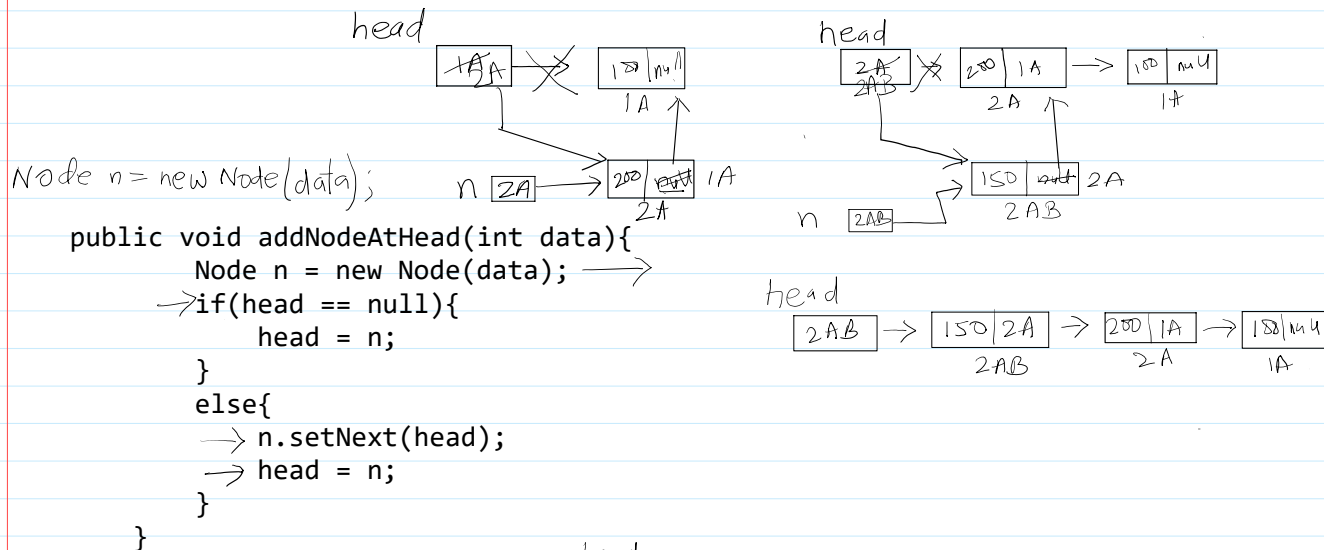
What is head?

Instead of keeping separate variables such as node1, node2 and node3. We generally maintain one reference variable called **head**.
head points to the first node.

Initially, head = null; which means the linked list is empty.



is deleted because local variables or objects are collected by garbage collector, when program control goes back to the caller of the method.



```

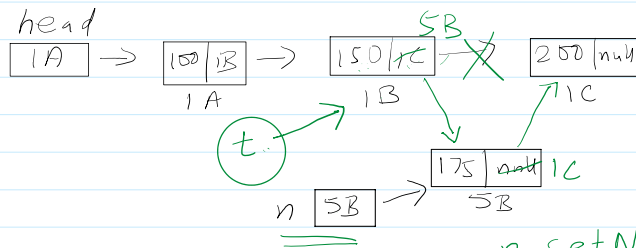
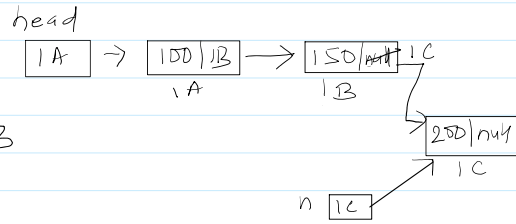
public void addNodeAtEnd(int data){
    head
    1A -> 100 1B -> 150 1C

```

```

public void addNodeAtEnd(int data){
    Node n = new Node(data);
    if(head == null){
        head = n;
    }
    else{
        Node t = head;
        //loop used to point the last node of the linked list
        for(;t.getNext() != null; t.setNext(t.getNext()));
        t.setNext(n); //attaching the node at end;
    }
}

```



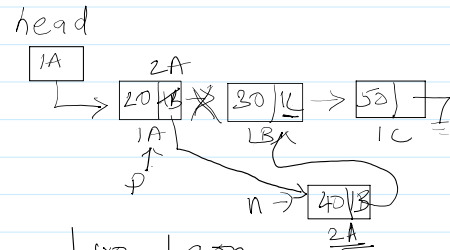
after = 150

$n.setNext(t.getNext());$
 $t.setNext(n);$

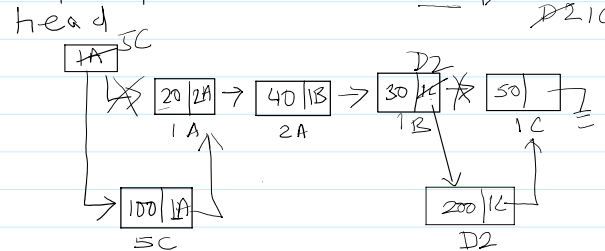
```

public void addNodeBeforeANode(int data, int before){
    if(head == null){
        System.out.print("\nList is empty");
        return;
    }
    for(Node p = null, t = head; t != null; t = t.getNext()){
        if(t.getData() == before){
            Node n = new Node(data);
            n.setNext(t);
            if(p == null){head = n; return;}
            p.setNext(n);
            return;
        }
        p = t;
    }
    System.out.print("\n" + before + " is not present in the list.");
}

```



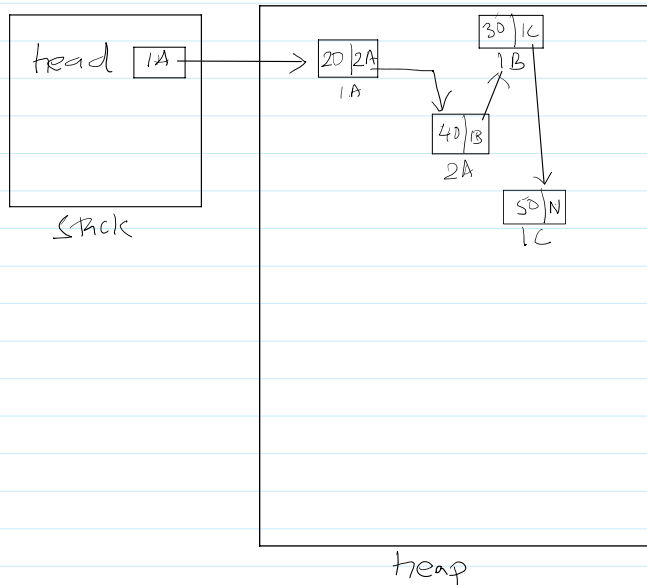
data: 40	180	200	500
before: 30	20	50	21
t = 1A	1A	5C 1A 2A 1B 1C	5C 1A 2A 1B 1C
n = 2A	5C	D2	5C 1A 2A 1B 1C
p = 1A	1A	5C 1A 2A 1B 1C	5C 1A 2A 1B 1C



✓ addNodeBeforeANode(100, 20);

✓ addNodeBeforeANode(200, 50);

addNodeBeforeANode(500, 21);



```

public void addNodeAfterANode(int data, int after) {
    → if(head == null) {
        System.out.print("\nList is empty");
        return; //exit from the method
    }
    for(Node t = head; t != null; t = t.getNext()) {
        if(t.getData() == after) {
            → Node n = new Node(data);
            n.setNext(t.getNext());
            → t.setNext(n);
            → return ; //exit from the method
        }
    }
    System.out.print("\n" + after + " is not present in the list.");
}

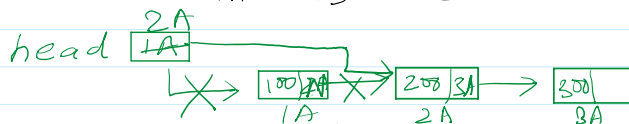
```

Linked list: ==>100 ==>10 ==>200 ==>30 ==>50 ==>500 ==>1000 ==>null

1A 1B 1C 1D 1E 1F 2A

head 2A

2A 2B



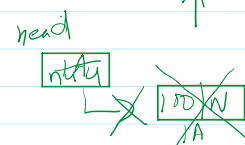
```

t = head;
→ head = head.getNext();
→ if (head != null) {
    → t.setNext(null)
}

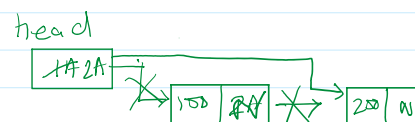
```

I need to disconnect the node from the list.

Java: all orphan object/local variables are clean up by garbage collector,

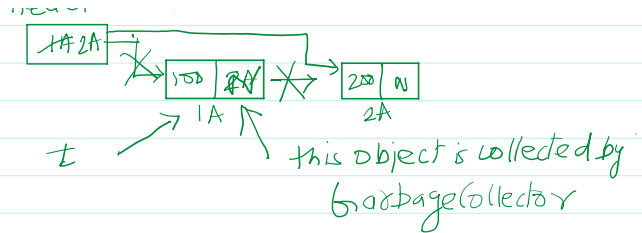


t = 1A



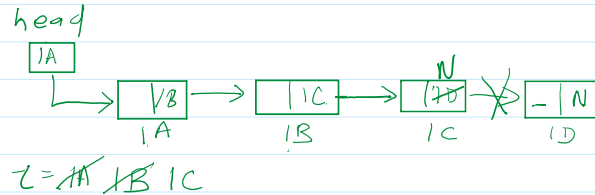
In Java, **garbage collection** is the process of automatically managing memory by reclaiming objects that are no longer referenced or needed

In Java, **garbage collection** is the process of automatically managing memory by reclaiming objects that are no longer referenced or needed by a program. This ensures efficient memory usage and prevents memory leaks. The Java Virtual Machine (JVM) handles this process, relieving developers from manually deallocating memory.



//Case of multiple nodes

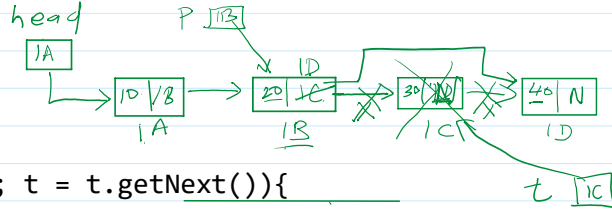
```
Node t = head;
while(t.getNext().getNext() != null){
    t = t.getNext();
}
System.out.print("\n Deleted Node: " + t.getNext().getData());
t.setNext(null);
```



```
Node p = null, t = head;
//Case of 1st node match.
```

```
if(t.getData() == data){
    if(head.getNext() == null){
        head = null;
        System.out.print("\nDeleted node successfully.");
        return;
    }
}
```

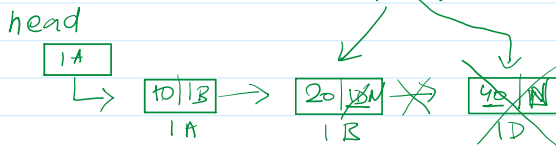
data = 30
t = 1A 1B 1C
p = null 1A 1B



//Case is in the middle or end

```
for(p = t, t = t.getNext(); t != null; t = t.getNext()){
    if(t.getData() == data){
        p.setNext(t.getNext());
        t.setNext(null);
        System.out.print("\nNode deleted successfully.");
        return;
    }
    p = t;
}
```

data = 40
t = 1A 1B 1D
p = null 1A 1B



Why use a LinkedList over an Array?

A linked list is a data structure that overcomes the limitations of arrays.

Let's first see some of the limitation of arrays:

- The size of the array must be known in advance before using it in the program.
- Increasing the size of the array is time - consuming process. It is almost impossible to expand the size of the array at runtime.
- All the elements in the array need to be contiguously stored in the memory. Inserting an element in the array needs shifting of all its predecessors.

A linked list is useful because:

- It allocates the memory dynamically. All the nodes of the linked list are

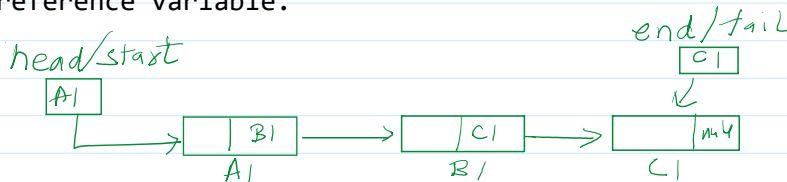
non - contiguously stored in the memory and linked together with the help of pointers or reference variables.

- In a linked list, size is no longer a problem since we do not need to define its size at the time of declaration. The list grows as per the program's demand and is limited to the available memory space.

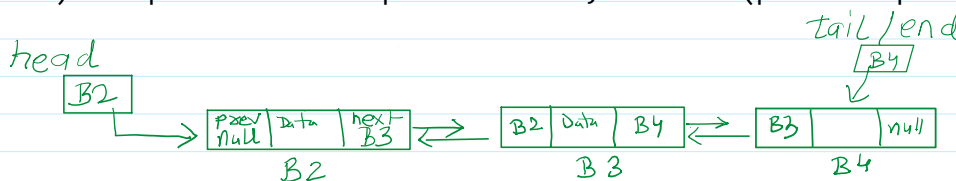
Types of Linked List

A link list is classified to the following types:

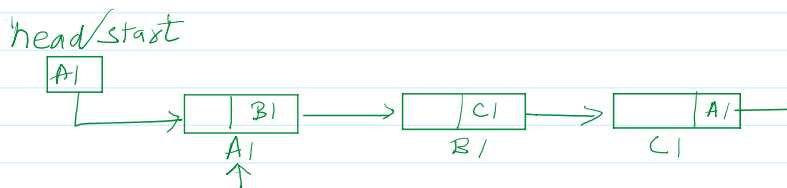
1. **Singly - Linked List:** It is the simple linked list, the first node is pointed by head node, and the next nodes reference or address is hold by next reference variable.



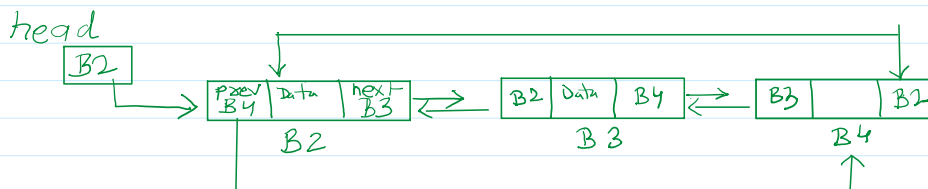
2. **Doubly Linked List:** A doubly linked list is a complex type of linked list in which a node contains A pointer to the previous as well as next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts, node data, pointer to the next node in sequence (next pointer) and pointer to the previous node, that is (previous pointer).



3. **Circular Linked List:** In a circular linked list, the last node of the list contains a pointer or reference to the first node of the list. We can have a circular singly linked list as well as a circular doubly linked list.



4. **Circular Doubly Linked List:** A circular doubly linked list is a more complex type of data structure in which a node contains pointer to the previous node as well as next node. A circular doubly linked list does not contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.



Advantages of Linked List

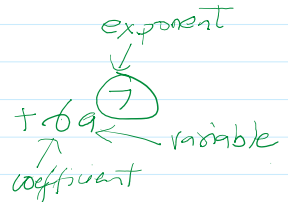
The advantages of using the linked list are given as follows:

- **Dynamic data structure:** The size of the linked list may vary according to the requirements. Linked list does not have a fixed size.
- **Insertion and deletion:** Unlike arrays, insertion and deletion in a linked list are, Aries elements are stored in consecutive locations, whereas the elements in the linked list are stored at random locations. To insert or delete an element in an array, we have to shift the elements to create space. Whereas in linked list instead of shifting we just have to update the address of the pointer or reference of the node.
- **Memory efficient:** The size of a linked list can grow or shrink according to the requirement. So the memory consumption in a linked list is efficient.
- **Implementation:** We can implement both stacks and queues using a linked list.

Disadvantages of Linked List

The limitations of using the linked list are given as follows:

- **Memory usage:** In a linked list, a node occupies more memory than an array element. Each node of the linked list occupies two types of variable. That is one is a simple variable and other is a reference variable or pointer variable.
- **Traversal:** Traversal is not easy in a linked list. If we have two axes and element in the linked list, we cannot access it randomly while in case of array we can randomly access it by index. For example, if we want to access the 3rd node then we need to traverse all the node before it. So the time required to access a particular node is large.
- **Reverse traversing:** Backtracking or reverse traversing is difficult in a linked list. In a doubly linked list it is easier but requires more memory to store the back pointer.



Application of Linked List:

- With the help of linked list, the polynomials can be represented as. We can perform the operations on the polynomial.
- A linked list can be used to represent the sparse matrix.
- The various operations like student details, employees details or product details can be implemented using the linked list. As a linked list uses the structured data type that can hold different data types.
- Using a link list, we can implement a stack, a queue, a tree and other various data structures.
- The graph is a collection of edges and vertices, and the graph can be represented as an adjacency matrix and an adjacency list. If we want to represent the graph as an adjacency matrix, then it can be implemented as an array. If we want to represent the graph as an adjacency list, then it can be implemented as a linked list.
- A linked list can be used to implement dynamic memory allocation. The dynamic memory allocation is the memory allocation done at runtime.

Homework:

Use the existing code, and implement end reference variable in Linked List class, and do the required changes in the methods of the class.

Add two more methods, which returns the head element and the other method will return end element.